

# METODOLOGÍA I

## Trabajo Práctico — El loop del agente en acción

### OBJETIVO GENERAL

Operar un agente de terminal real sobre una tarea propia con un objetivo verificable, documentando el loop completo observar-planificar-actuar-verificar, configurando su harness mediante un archivo de reglas ( `AGENTS.md` ), y clasificando la arquitectura (reactiva o deliberativa) que se observó durante la ejecución.

### MARCO TEÓRICO

| Concepto de la unidad                     | Aplicación en este TP                                                                                                                                 |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Objetivo verificable                      | Elegís una tarea con una condición de verdad medible (tests pasan, linter en 0 errores, archivos con un contenido exacto) antes de empezar.           |
| Loop observar-planificar-actuar-verificar | Trazás cada vuelta del loop del agente sobre tu tarea, tal como se hizo con el caso "sin errores de lint" en la Actividad 1.                          |
| Tool Calling                              | Identificás qué herramienta concreta usó el agente en cada acción (leer archivo, ejecutar comando, editar, buscar texto).                             |
| Harness (Agente = Modelo + Harness)       | Escribís un <code>AGENTS.md</code> mínimo con reglas fijas para tu proyecto y verificás que el agente las cargue sin que se las repitas en el prompt. |
| Arquitecturas de agentes                  | Justificás si el comportamiento observado fue reactivo (regla fija) o deliberativo (evaluó alternativas antes de actuar).                             |

### PARTE A — TRAZADO DEL LOOP EN UNA TAREA REAL

Elegí una tarea real sobre un proyecto propio (un TP anterior, un ejercicio del cuadernillo, o un repositorio personal) que tenga un **objetivo verificable**: por ejemplo, "dejar el proyecto sin errores de lint", "hacer que pasen todos los tests", o "crear una estructura de 3 archivos con un contenido exacto". Resolvela con un agente de terminal (Claude Code, Antigravity, Cursor u otro) y documentá lo siguiente.

#### 1. Objetivo elegido y por qué es verificable

Escribí el objetivo tal cual se lo diste al agente, y explicá con qué condición de verdad medible vas a comprobar que se cumplió.

## 2. Tabla del loop

Reconstruí, vuelta por vuelta, lo que hizo el agente en una tabla con estas columnas: **Vuelta, Observar, Planificar, Actuar (herramienta usada), Verificar** — igual que la tabla del ejercicio 4.1 de la Unidad 4, pero con tu propio caso.

## 3. Herramientas (Tool Calling)

Listá cada herramienta distinta que usó el agente durante la tarea (por ejemplo: leer archivo, ejecutar comando, editar, buscar texto) y para qué la usó.

## 4. Clasificación de arquitectura

¿El agente actuó de forma reactiva (regla fija, sin evaluar alternativas) o deliberativa (mantuvo un modelo del problema y decidió un orden o una estrategia)? Justificá con un momento concreto de tu propia traza.

## PARTE B — EL HARNESS A MEDIDA: TU AGENTS.MD

Escribí tu propio `AGENTS.md` en la raíz del proyecto que usaste en la Parte A, con un mínimo de 5 reglas fijas (mezclá idioma, estilo, prohibiciones y comportamiento ante casos específicos — nada de reglas vagas tipo "sé prolijo").

### 1. Test A — el agente cumple

Abrí una conversación nueva (para que el harness cargue el `AGENTS.md`) y dale una instrucción cualquiera. Documentá qué regla se notó cumplida sin que la repitieras en el prompt.

### 2. Test B — intento de violación

Pedile al agente algo que choque con una de tus reglas (por ejemplo, si tu regla dice "pedí confirmación antes de borrar archivos", pedile que borre algo sin confirmar). Documentá si la regla se respetó o no.

### 3. Reflexión breve

Si en el Test B la regla NO se respetó, ¿qué le cambiarías al texto de la regla para que fuera más clara? Si Sí se respetó, ¿qué le aportó concretamente el `AGENTS.md` frente a tener que repetir esa regla en cada prompt?

## PARTE C — REFLEXIÓN FINAL

**Reflexión final (mínimo 100 palabras):** tu tarea de la Parte A la resolvió un único agente. Imaginá que la tarea fuera mucho más grande —por ejemplo, agregar una funcionalidad completa nueva a un proyecto, no solo corregir errores de lint—. ¿Por qué un agente único empezaría a fallar al manejar tantos criterios a la vez? Cerrá anticipando qué problema resuelve dividir el trabajo entre varios agentes especializados, tema de la Semana 5.

## CONSIDERACIONES

- ✓ **Elección de la tarea:** podés reusar un proyecto de un TP anterior; lo importante es que el objetivo tenga una condición de verdad medible, no que el proyecto sea nuevo.
- ✓ **Traza completa:** una tabla del loop con una sola fila, o sin la columna de herramienta usada, se considera incompleta.
- ✓ **AGENTS.md real:** las reglas deben ser específicas y verificables (negativas o condicionales), no genéricas tipo "sé prolijo" o "respondé bien".
- ✓ **Test B sin retoque:** si la regla no se respetó, documentalo tal cual — es información válida, no un error a esconder.

## CONCLUSIONES ESPERADAS

- Reconocer el loop observar-planificar-actuar-verificar en una tarea propia, no solo en el ejemplo de clase.
- Configurar un harness mínimo propio ( `AGENTS.md` ) y verificar empíricamente qué le aporta frente a repetir instrucciones en cada prompt.
- Justificar con evidencia concreta si un comportamiento observado fue reactivo o deliberativo.
- Llegar a la Semana 5 con la pregunta ya planteada: qué pasa cuando un agente único no alcanza para manejar todos los criterios de una tarea grande.